Livrable E2

Veille technique et règlementaire

Franck Le Fur

Le 30 Mars 2026

Titre professionnel **Développeur en intelligence artificielle**
de niveau 6 enregistré au RNCP sous le n°37827.

Passage par la voie de la formation - parcours de 19 mois achevé en mars 2026

Table des matières

1 Présentation **3**
1.1 Instructions du projet / Objectifs du Rapport . . . 3
1.2 Présentation du projet . . . 3
1.3 Introduction . . . 3

2 Organisation de la veille technlogique **5**
2.1 Les sources . . . 5
2.2 Automatiser la collecte des sources . . . 5
2.2.1 L'API Arxiv . . . 5
2.2.2 Les alertes Google Scholar . . . 6

3 Représentations Discrètes et Fréquentielles **7**
3.1 Encodage One-Hot . . . 7
3.2 Le Modèle Bag-of-Words (BoW) . . . 7
3.3 TF-IDF (Term Frequency-Inverse Document Frequency) . . . 9
3.4 Avantages de TF-IDF par rapport à BoW . . . 9
3.5 Limitation . . . 9

4 Les Plongements Lexicaux Statiques et Prédictifs (Word Embeddings) **11**
4.1 Word2Vec : Architectures Neuronales Légères . . . 11
4.2 GloVe (Global Vectors for Word Representation) . . . 12
4.3 FastText : Robustesse par les Sous-Mots (Subword) . . . 13
4.4 La Limite de la Sémantique Statique . . . 14

5 L'Encodage Contextuel et des Transformers **15**
5.1 L'Architecture Transformer : Attention et Encodage Positionnel . . . 15
5.2 BERT (Bidirectional Encoder Representations from Transformers) . . . 15
5.3 GPT (Generative Pre-trained Transformer) . . . 15
5.4 La Divergence Fondamentale : Encodage vs. Génération . . . 16

6 Services d'intelligence artificielle pour de l'embedding **17**
6.1 Mistral . . . 17
6.2 Cohere . . . 17
6.3 Hugging Face . . . 18
6.4 OpenRouter . . . 19

7 Évaluation des embeddings **20**
7.1 Simple évaluation - Qualité sémantique . . . 20
7.2 Evaluation académique - Dataset MTEB . . . 20
7.3 Les Benchmark MTEB . . . 20
7.4 Les Benchmarks Standards : GLUE et SuperGLUE . . . 20

8 Défis Opérationnels, Éthiques et Perspectives d'Avenir **22**
8.1 Le Défi du Coût de Calcul . . . 22
8.2 Défis Éthiques : La Propagation des Biais . . . 22
8.3 Tendances Futures : Les LLMs Multimodaux . . . 22

9 Conclusion **23**

10 Divers Infos **24**
10.1 Méthodes basées sur le texte brut 25
10.2 Méthodes statistiques (bag-of-words) 25
10.3 Méthodes basées sur les embeddings 26
10.4 Méthodes avec Transformers (modèles contextuels) 26
10.5 Divers 26
10.6 Principe Sémantique 26

11 Latex Tools **28**

1 Présentation

1.1 Instructions du projet / Objectifs du Rapport

Un travail de recherche approfondi est demandé, listez les modèles utilisés, l'historique du sujet, les défis industriels que le domaine est susceptible de résoudre.
Pour chaque modèle réalisez une recherche complète :

- Expliquer les métriques utilisées pour tester et améliorer les modèles
- Listez les modèles/services déjà disponibles, comment ont-ils été entraînés, quelle volumétrié de données ?
- Expliquez les architectures utilisées, les fonction de perte, les techniques permettant de réduire l'overfitting etc.. Quelles méthodes récentes ont été utilisées ?
- Quelles sont les limites de chaque modèle/solution en place ?
- Quelles perspectives dans le futur ?
- Quels moyens sont nécessaires pour entraîner/faire l'inférence ?
- Présenter un panorama exhaustif des techniques d'encodage.
- Expliquer les principes, avantages et inconvénients de chaque méthode.
- Montrer l'évolution des approches (des méthodes "bag-of-words" aux embeddings contextuels).
- Aider à choisir la méthode adaptée à une tâche donnée.

1.2 Présentation du projet

Dans le cadre de mon sujet de société, il m'a été demandé de créer un moteur de recherche sur l'ensemble de nos documents internes (procédures, process techniques, bagages de formation, supports pédagogiques ...). Le traitement du langage naturel est au coeur d'un tel sujet et la question de l'encodage des textes en nombre s'est alors naturellement posée (aussi appelée vectorisation).
Dans cette veille nous ferons un état des lieux des méthodes d'encodage existantes et nous les évaluerons sur les Dataset classiques MTEB incluant différents cas d'usage.

1.3 Introduction

Les ordinateurs travaillent exclusivement avec des 0 et des 1, si la conversion des nombres vers une représentation binaire est naturelle, la consersion des données non-structurées en nombres est plus compliquée. Comment alors travailler avec des images ou du texte ?

La conversion d'images en nombres se fait assez naturellement si l'on considère une image comme un tableau de pixels et chaque pixel comme un vecteur numérique correspondant à la couleur RGB, la transparence etc ...

La vectorisation du texte est moins naturelle, il existe un grand nombre de techniques, dans ce document nous étudierons les méthodes basées sur l'encodage discret ou fréquentiel, les plongements lexicaux (embedding) et les méthodes basées sur l'encodage contextuel avec les transformers.
L'encodage du texte en vecteurs numériques est l'étape indispensable qui permet aux ordinateurs de traiter et d'interpréter le langage humain et ainsi de réaliser des tâches comme par exemple :

- Classification de texte (analyse des sentiments, la détection des spams, et la catégorisation des sujets).

— Traduction de textes.
— Extraction d'informations (reconnaissance d'entités nommées.
— Génération.
— Recherche d'informations : L'encodage en vecteurs permet la recherche vectorielle, qui améliore la mise en correspondance des requêtes utilisateur avec les documents pertinents. (C'est ce dernier point qui sera utilisé dans notre projet de société).

L'efficacité des modèles de Machine Learning à réaliser ces tâches réside dans la capacité de l'encodage à comprendre **la syntaxe du langage** (loi qui régit l'agencement et la structure des mots dans les phrases) et à **capturer leur signification et leurs relations sémantiques**.

2 Organisation de la veille technlogique

2.1 Les sources

Pour réaliser cette veille technique sur les méthodes de vectorisation de documents, nous consultons des sites tels que : GeeksForGeeks, Archiv.org, Google Scholar.

GeeksForGeeks est utilisé pour collecter des informations sur les méthodes "anciennes", par exemple les méthodes basées sur la fréquence des mots comme Bag-Of-Words, TF-IDF ou les méthodes basées sur le plongement lexical (GLOVE, Word2Vec), Au contraire les sources comme Archiv.org et Google Scholar sont utilisées pour les méthodes plus récentes (transformers) et pour se maintenir à jour des dernières évolutions.

2.2 Automatiser la collecte des sources

2.2.1 L'API Arxiv

Le site **Arxiv.org** offre une API, laquelle peut être requêter à l'aide d'un module python,

```python
import arxiv

search = arxiv.Search(
    query="embedding",
    max_results=5,
    sort_by=arxiv.SortCriterion.SubmittedDate
)

for paper in search.results():
    print(f"Titre : {paper.title}")
    print(f"Auteurs : {paper.authors}")
    print(f"Resume : {paper.summary[:200]}...")
    print(f"Lien : {paper.pdf_url}\n")
```

```
Titre : STEM: Scaling Transformers with Embedding Modules
Auteurs : [arxiv.Result.Author('Ranajoy Sadhukhan'), arxiv.Result.Author('Sheng Cao'), arxiv.Result.Author('Harr
Résumé : Fine-grained sparsity promises higher parametric capacity without proportional per-token compute, but c
Lien : https://arxiv.org/pdf/2601.10639v1

Titre : Circumplanetary Disk Candidate in the Disk of HD 163296 Traced by Localized Emission from Simple Organic
Auteurs : [arxiv.Result.Author('Andres F. Izquierdo'), arxiv.Result.Author('Jaehan Bae'), arxiv.Result.Author('M
Résumé : Atacama Large Millimeter/submillimeter Array observations suggest that the disc of HD 163296 is being a
Lien : https://arxiv.org/pdf/2601.10631v1

Titre : RSATalker: Realistic Socially-Aware Talking Head Generation for Multi-Turn Conversation
Auteurs : [arxiv.Result.Author('Peng Chen'), arxiv.Result.Author('Xiaobao Wei'), arxiv.Result.Author('Yi Yang'),
Résumé : Talking head generation is increasingly important in virtual reality (VR), especially for social scenar
Lien : https://arxiv.org/pdf/2601.10606v1

Titre : Institutional AI: A Governance Framework for Distributional AGI Safety
Auteurs : [arxiv.Result.Author('Federico Pierucci'), arxiv.Result.Author('Marcello Galisai'), arxiv.Result.Autho
Résumé : As LLM-based systems increasingly operate as agents embedded within human social and technical systems,
Lien : https://arxiv.org/pdf/2601.10599v1
```

FIGURE 1 – Résultats d'une requête de l'API Arxiv.com

En production, on pourrait automatiser ce genre de requêtes avec une tâche crontab et envoyer un email avec les résultats obtenus.

2.2.2 Les alertes Google Scholar

Le site **Google Scholar** propose un système d'alerte en fonction de mots-clés, nous prévenant par email dès qu'un article correspondant à nos critères vient d'être publié.

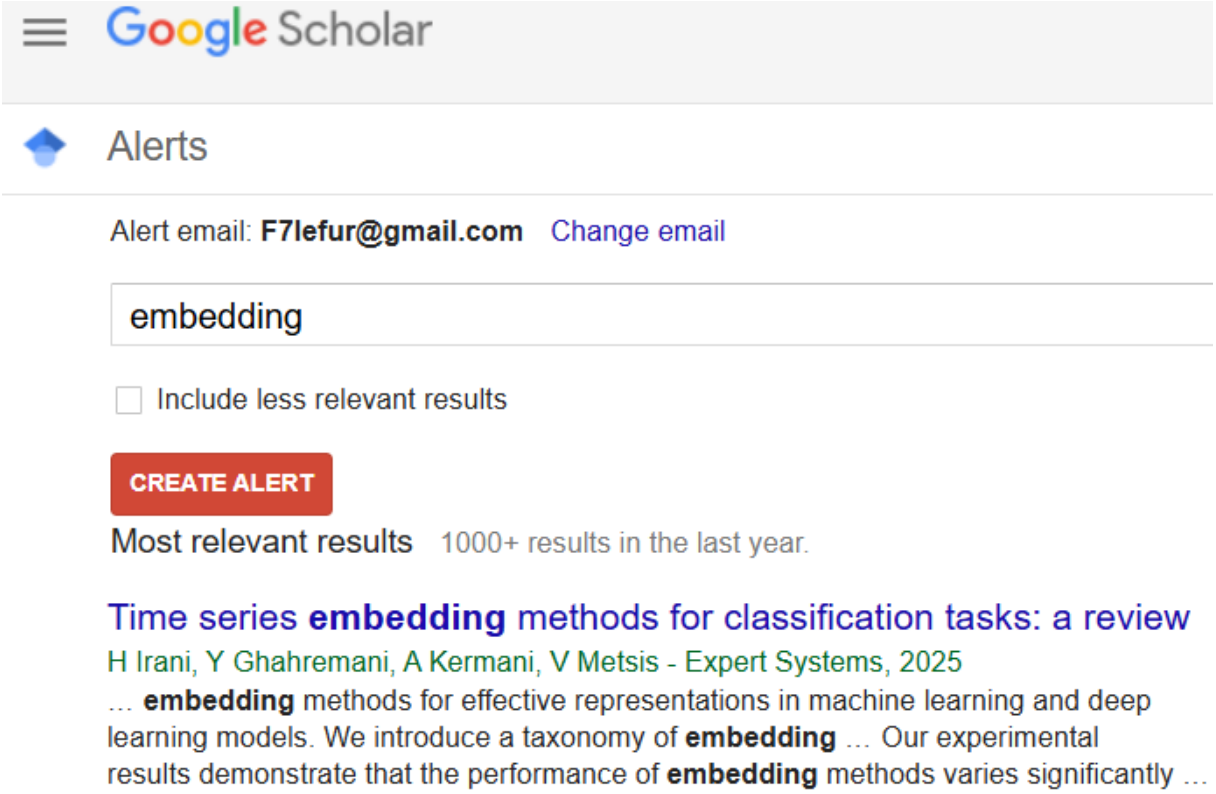


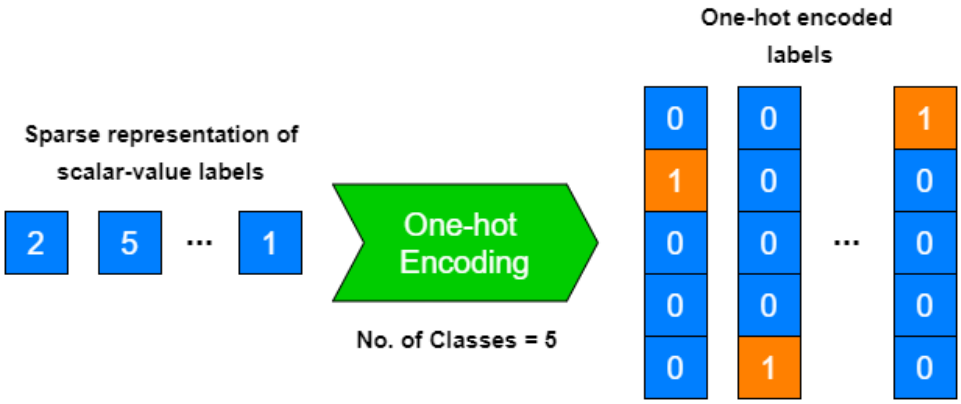
FIGURE 2 – Alertes automatiques avec Google Scholar

3 Représentations Discrètes et Fréquentielles

Cette première ère de vectorisation se caractérise par l'utilisation de représentations "creuses" (sparse), où la dimensionnalité est proportionnelle à la taille du vocabulaire et où la grande majorité des valeurs vectorielles est nulle.

3.1 Encodage One-Hot

Le One-Hot Encoding est l'approche la plus élémentaire. Chaque mot unique du vocabulaire en encodé en un vecteur binaire, contenant un 1 à la position correspondant à ce mot spécifique et des 0 partout ailleurs.



Inconvénients :

- Les vecteurs sont de dimensions trop grandes (de dimension la taille du dictionnaire),
- Cette méthode ne permet pas au modèle de comprendre la syntaxe du langage,
- Cette méthode ne permet pas de capturer des relations sémantiques entre les mots, c'est-à-dire que 2 mots sémantiquement proches auront des représentations éloignées. (Mathématiquement, le produit scalaire entre deux vecteurs One-Hot différents est toujours nul, ce qui signifie que toutes les paires de mots sont considérées comme orthogonales, on ne peut donc pas tenir compte du calcul de similarité cosinus).

Utilisation : Encoder des variables catégorielles

3.2 Le Modèle Bag-of-Words (BoW)

Le modèle Bag-of-Words (BoW), ou "Sac de Mots," encode des textes en se basant sur l'occurence des mots.
Fonctionnement :

- Création du vocabulaire à partir des mots uniques d'un ensemble de documents,
- Calcul de l'occurrence de chaque mot du vocabulaire dans l'ensemble des documents,
- Chaque document est représenté par un vecteur où chaque nombre correspond à l'occurrence d'un mot dans le document.

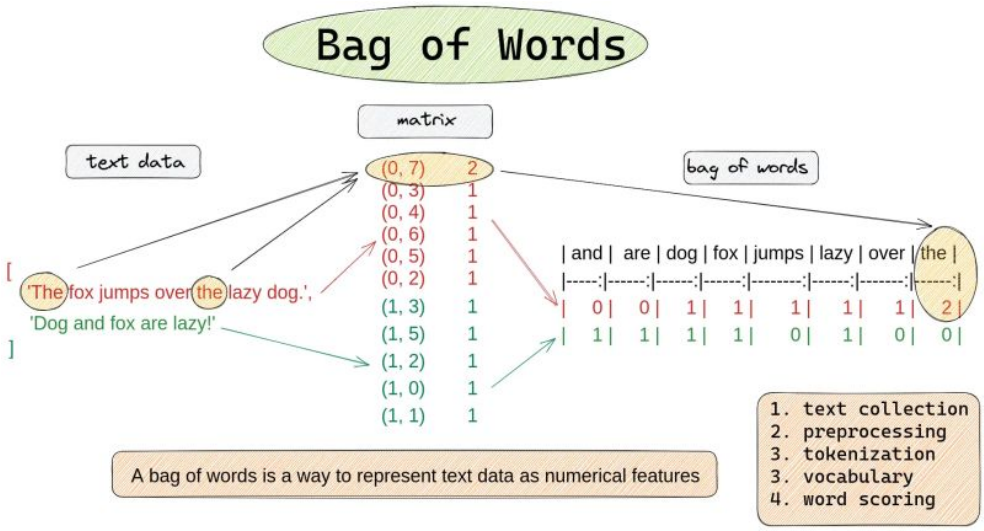


FIGURE 3 – Bag-of-Words

Inconvénients :

— Dimension des vecteurs trop importante,

— Ne comprend pas la syntaxe du langage,

— Ne capture pas la sémantique des mots ou des phrases, elle peut encoder de la même manière des phrases différentes. (Elle ne tient pas compte de l'ordre des mots dans une phrase).

— Il est difficile d'ajouter de nouveaux mots.

Document D1	*The child makes the dog happy* the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	*The dog makes the child happy* the: 2, child: 1, makes: 1, dog: 1, happy: 1

↓

	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

FIGURE 4 – Bag-of-Words

La méthode Bag-of-Words peut être utilisée pour des tâches simples de classification de textes, les calculs sont très rapides.

3.3 TF-IDF (Term Frequency-Inverse Document Frequency)

La méthode TF-IDF améliore le modèle Bag of Words en transformant le simple "comptage" de mots en une mesure de l'importance relative de chaque mot au sein d'un document et d'un corpus.
La méthode combine deux facteurs :

— Term Frequency (TF) : La fréquence d'apparition d'un terme dans un document.

— Inverse Document Frequency (IDF) : L'inverse de la fréquence du document, mesurant à quel point un terme est rare ou commun à travers l'ensemble du corpus.

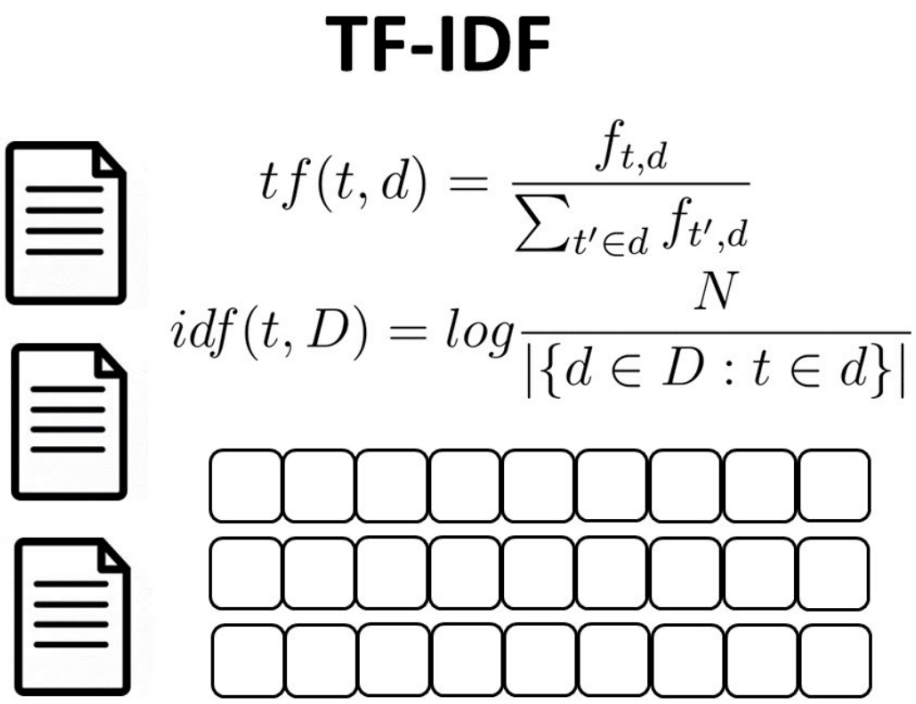


Figure 5 – TF-IDF

Le score TF-IDF est le produit de ces deux composantes. Ce mécanisme réduit l'importance des mots trop courants (par exemple les stop words) qui apparaissent dans presque tous les documents. Par conséquent, TF-IDF

3.4 Avantages de TF-IDF par rapport à BoW

BoW apporte trop d'importance aux mots trop courants (par exemple les stop words), le terme IDF permet de résoudre ce problème (on parle de réduction du bruit). A l'inverse un mot rare dans le corpus de document mais présent dans un document reçoit un score plus élevé (meilleure valorisation des mots discriminants).

En conséquences les algorithmes de Machine Learning seront plus efficaces.

3.5 Limitation

Toutes ces méthodes discrètes basées sur la fréquence des mots présentent deux inconvéninents majeurs :

— La haute dimensionnalité des vecteurs creux freinent leur utilisation dans le contexte du Deep Learning (on cherche toujours à réduire la dimension des données).

— Elles ne permettent pas de capturer la structure grammaticale du langage ni de capturer la sémantique des mots. Par exemple, statistiquement, les mots "voiture" et "automobile" sont traités comme deux entités distinctes bien que leur signification soit quasi identique.

what-is-bm25-best-matching-25-algorithm

4 Les Plongements Lexicaux Statiques et Prédictifs (Word Embeddings)

Dans ces méthodes, les représentations vectorielles seront "denses", de faibles dimensions et avec des valeurs continues.

4.1 Word2Vec : Architectures Neuronales Légères

Word2Vec repose sur l'idée que les mots apparaissant dans des contextes similaires ont des significations proches et que leurs vecteurs doivent être géométriquement proches dans l'espace vectoriel, ce qui capturerait la similarité sémantique et syntaxique des textes. Word2Vec consiste en 2 architectures principales **CBOW (Continuous Bag of Words)** et **Skip-gram** qui utilisent des réseaux de neurones peu profonds (deux couches) pour apprendre des vecteurs en reconstruisant les contextes linguistiques des mots.

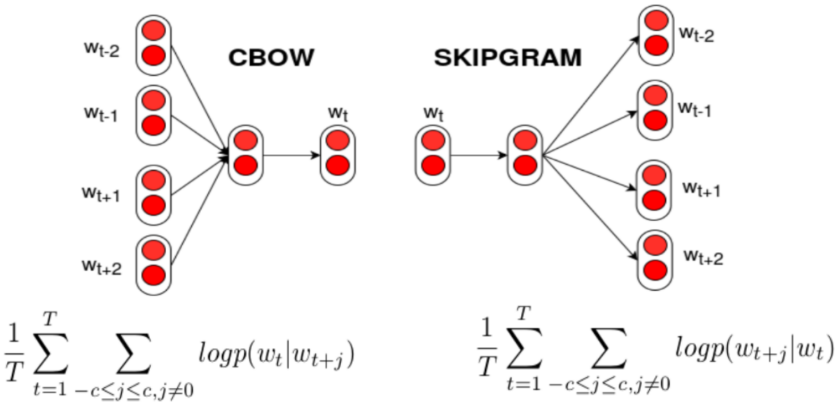


FIGURE 6 – Architectures CBOW et Skip-gram

Dans l'approche CBOW, la tâche consiste à prédire le mot central à partir de la somme vectorielle de ses mots voisins dans une fenêtre de contexte donnée. (tâche de remplissage du blanc)
Le modèle Skip-gram inverse la tâche de CBOW : il utilise le mot central pour prédire les mots qui l'entourent (son contexte).

Remarques

- CBOW est généralement plus rapide à entraîner que Skip-gram,
- CBOW est plus efficace pour les mots fréquentes,
- Skip-gram est plus efficace pour obtenir des représentations de qualité pour les mots peu fréquents,
- Word2Vec capture la sémantique des mots (les représentations vectorielles de **chat** et **chaton** vont être proches au sens de la similarité cosinus),
- Word2Vec capture la syntaxe du langage (compréhension du pluriel, du genre, superlatif, contraire etc ...),
- Word2Vec permet de construire une arithmétique sur les mots par exemple : "Le roi - homme + femme = la reine".

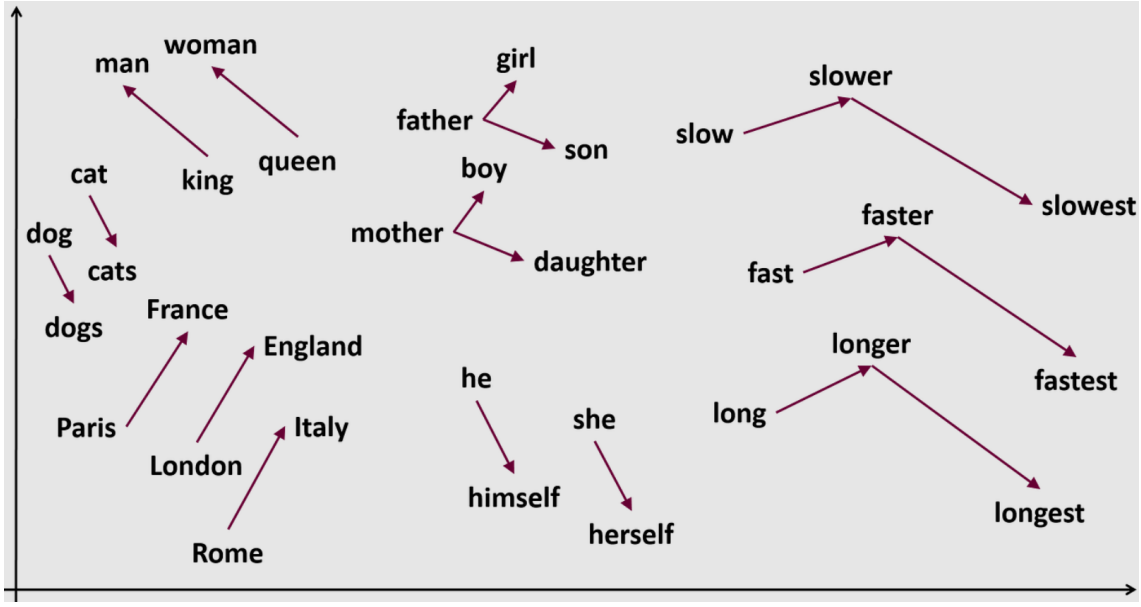


FIGURE 7 – Géométrie avec Word2Vec

4.2 GloVe (Global Vectors for Word Representation)

Objectif : capturer la sémantique lexicale en apprenant des représentations vectorielles denses (embeddings) des mots, de telle sorte que :

Les mots sémantiquement proches aient des vecteurs proches (en similarité cosinus ou distance euclidienne). Les relations linéaires entre mots (ex. analogies) soient approximativement conservées dans l'espace vectoriel. Contrairement à Word2Vec (qui est basé sur des contextes locaux), GloVe exploite les statistiques globales de co-occurrence dans un corpus entier.

GloVe part d'une observation simple mais puissante : Le rapport des probabilités de co-occurrence peut révéler des relations sémantiques.

Par exemple : P(solide | glace) / P(solide | vapeur) est élevé. P(gazeux | glace) / P(gazeux | vapeur) est faible.

Ces rapports permettent de distinguer des concepts même si les fréquences absolues sont différentes.

L'idée : on veut que la différence entre les vecteurs de mots encode ces rapports de probabilités.

Le processus repose sur l'analyse de la fréquence à laquelle les mots apparaissent ensemble dans un large corpus de texte.

Matrice de cooccurrence : L'algorithme construit une matrice mondiale où chaque élément indique combien de fois une paire de mots apparaît à proximité l'une de l'autre dans une fenêtre de contexte définie. Optimisation mathématique : GloVe optimise les vecteurs de sorte que leur produit scalaire soit lié aux probabilités de cooccurrence des mots (plus précisément à l'Information Mutuelle Ponctuelle ou PMI).

Comptage des Co-occurrences : La matrice de co-occurrence X est construite, où X_{ij} est le nombre de fois où le mot j apparaît dans le contexte du mot i.3Modélisation Probabiliste : Le modèle cherche à apprendre deux vecteurs, w_i et $\tilde{w}_i$, pour chaque mot i, de sorte que leur produit scalaire, ajusté par des biais, approxime le logarithme de la probabilité de co-occurrence $P_{ij} = P(j|i)$. La relation clé est la régression log-bilinéaire :

$$w_i^T \tilde{w}_j + b_i + \tilde{b}_j \approx \ln P_{ij}$$

.

Optimisation : L'apprentissage est effectué en minimisant une fonction de perte quadratique pondérée. La pondération est cruciale pour que les co-occurrences fréquentes aient plus d'influence que les co-occurrences rares et potentiellement bruitées. La fonction de perte minimisée est :

$$L = \sum_{i,j \in V} f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \ln X_{ij})^2$$

. Une fois entraîné, la représentation finale recommandée pour le mot i est la somme de ses deux vecteurs, $w_i + \tilde{w}_i$.

Entraînement Prétraitement : Tokenisation, construction du vocabulaire (souvent limité aux mots les plus fréquents, ex. 40k–100k). Calcul de la matrice de co-occurrence X (en pratique, on ne stocke pas toute la matrice, mais on utilise des structures efficaces). Optimisation : Minimisation de J via descente de gradient stochastique (SGD). Contrairement à Word2Vec (qui utilise des échantillons de contextes), GloVe utilise directement les statistiques globales, donc l'entraînement est plus stable. Vecteur final : Après entraînement, on combine souvent w_i et $\tilde{w}_i$ (ex. moyenne ou somme) pour obtenir une seule représentation par mot.

Avantages Contrairement à Word2Vec (local), GloVe voit tout le corpus → meilleure estimation des fréquences rares. Performances solides Très compétitif sur les tâches de similarité et d'analogie. Entraînement stable Moins sensible au bruit aléatoire (pas de sampling de négatifs). Open-source Implémentations officielles disponibles (C, Python).

Limites Statique : un mot a toujours la même représentation, même dans des contextes différents (ex. "banque" = rivière vs institution financière). Ne gère pas les mots hors vocabulaire (OOV). Moins performant que les modèles contextuels modernes (ELMo, BERT, etc.) sur des tâches complexes.
Source : https://www.geeksforgeeks.org/nlp/glove-word-embedding-in-nlp/

4.3 FastText : Robustesse par les Sous-Mots (Subword)

Le problème des méthodes comme Word2Vec et GloVe est leur incapacité à générer des vecteurs pour les mots hors-vocabulaire, car ils attribuent un vecteur unique à chaque mot entier vu pendant l'entraînement. FastText surmonte cette limitation en utilisant une approche basée sur les sous-mots (ou n-grammes de caractères). Si un mot hors-vocabulaire ou rare est rencontré, FastText génère son plongement en sommant les vecteurs des n-grammes de caractères qui le composent.
Objectif principal de FastText : Apprendre des représentations vectorielles denses des mots qui :

Capturent la sémantique (comme Word2Vec ou GloVe), Mais qui peuvent aussi généraliser aux mots inconnus (hors vocabulaire), En exploitant la structure interne des mots via des n-grammes de caractères.

Cette approche offre deux avantages majeurs :

Gestion OOV : Même un mot entièrement nouveau peut être encodé de manière significative si ses fragments (ses n-grammes) ont été vus pendant l'entraînement.

Morphologie et Rareté : FastText est particulièrement robuste pour les langues riches en morphologie (où les mots changent beaucoup par flexion ou conjugaison) et pour les fautes de frappe ou les mots rares, car il exploite l'information au niveau des caractères.

Chaque n-gramme a son propre vecteur d'embedding.

Le vecteur final du mot "where" est la moyenne des vecteurs de tous ses n-grammes + le vecteur du mot entier (optionnel).

Avantages Explication Gestion des mots rares Même si un mot apparaît peu, ses sous-mots (ex. "tion", "ing") sont partagés avec d'autres mots → meilleure généralisation. Prise en charge

des mots OOV (hors vocabulaire) Un mot jamais vu peut être encodé à partir de ses n-grammes (ex. "unfriendable" → "un", "fri", "rie", etc.). Efficace pour les langues morphologiques Langues comme le finnois, le turc ou l'arabe, où les mots changent beaucoup via affixes. Compatibilité avec Word2Vec Les embeddings produits sont interchangeables avec ceux de Word2Vec.

Limites Taille du modèle plus grande : il faut stocker des vecteurs pour tous les n-grammes (ex. centaines de milliers). Moins efficace pour les mots très courts (ex. "a", "I") : peu de sous-mots utiles. Toujours un embedding statique : pas de contexte dynamique (comme BERT). Sensible au bruit orthographique : fautes de frappe peuvent générer des n-grammes inconnus.

Cas d'usage typiques Classification de texte avec peu de données (grâce à la généralisation des subwords). Langues à faibles ressources ou avec morphologie complexe. Applications embarquées : FastText est léger et rapide en inférence. Correction orthographique / reconnaissance de néologismes.

4.4 La Limite de la Sémantique Statique

Malgré les avancées de Word2Vec, GloVe et FastText en termes de similarité sémantique, ils partagent un défaut fondamental : ils sont statiques. Chaque mot se voit attribuer un vecteur unique, ignorant le contexte spécifique de son apparition.

Cette contrainte empêche de distinguer des représentations pour le mot "avocat" selon que l'on parle du fruit ou de la fonction. Les modèles statiques ne peuvent représenter que la "sémantique moyenne" du mot dans le corpus. La nécessité de générer un vecteur dynamique pour chaque instance d'un mot, en fonction de son environnement, a conduit à l'avènement des modèles contextuels basés sur les Transformers.

5 L'Encodage Contextuel et des Transformers

L'introduction de l'architecture Transformer a marqué la troisième ère de la vectorisation, permettant des représentations dynamiques et contextuelles.

5.1 L'Architecture Transformer : Attention et Encodage Positionnel

Le Transformer a révolutionné le TALN en s'affranchissant des réseaux neuronaux récurrents (RNNs), qui traitaient l'information séquentiellement. Le cœur du Transformer est le mécanisme d'auto-attention (Self-Attention), qui permet au modèle de pondérer l'importance de tous les autres mots de la séquence lors du traitement d'un mot donné, se concentrant ainsi sur les parties pertinentes du texte.

Puisque le mécanisme d'auto-attention traite les mots simultanément, l'information inhérente à l'ordre séquentiel est perdue. Pour remédier à cela, l'encodage positionnel est intégré à l'architecture. Il s'agit d'un ajout de vecteurs qui introduit la notion de position relative et absolue des mots dans la séquence, information vitale pour la compréhension de la syntaxe et du contexte.

5.2 BERT (Bidirectional Encoder Representations from Transformers)

Développé par Google en 2018, BERT est un modèle pré-entraîné qui utilise l'encodeur du Transformer. Sa principale innovation est son caractère bidirectionnel : il analyse le contexte d'un mot en regardant simultanément ce qui précède (contexte gauche) et ce qui suit (contexte droit).

4.2.1. Résolution de la Polysémie La bidirectionnalité confère à BERT une capacité de compréhension contextuelle complète pour chaque mot. Contrairement aux modèles statiques, BERT génère un vecteur unique et dynamique pour chaque apparition du mot dans une phrase donnée. Par conséquent, il peut attribuer des vecteurs différents au mot "banque" selon qu'il apparaît dans le contexte de "bord de la rivière" ou "ouverture de compte," résolvant ainsi la polysémie.

4.2.2. Tâches de Pré-entraînement BERT est pré-entraîné sur deux tâches principales pour maximiser sa capacité de compréhension :

Masked Language Model (MLM) : Le modèle doit prédire des mots masqués dans la phrase à partir du contexte environnant.

Next Sentence Prediction (NSP) : Le modèle doit prédire si une phrase B suit logiquement une phrase A. Cela permet au modèle de comprendre les dépendances et les relations qui existent entre des phrases consécutives.

Grâce à ces innovations, BERT a rapidement établi des performances de pointe (SOTA, State-of-the-Art) sur de nombreux benchmarks de compréhension (GLUE, SQuAD). Des variantes ont été développées pour optimiser les performances ou les ressources, telles que RoBERTa (optimisation de l'entraînement), DistilBERT (version allégée, 40% plus rapide pour 97% des performances) et CamemBERT (modèle spécifiquement entraîné pour la langue française).

5.3 GPT (Generative Pre-trained Transformer)

Les modèles de la série GPT, développés par OpenAI, sont des modèles génératifs basés sur le décodeur du Transformer. Leur fonctionnement est intrinsèquement unidirectionnel : ils regardent uniquement le contexte précédent (gauche) pour prédire le mot suivant.

Cette structure unidirectionnelle les rend excellents pour la génération de texte cohérent, créatif et dans des styles variés , mais moins aptes à la compréhension discriminative fine que

les modèles bidirectionnels comme BERT. GPT est également reconnu pour sa polyvalence et sa capacité à effectuer du few-shot learning (apprentissage avec très peu d'exemples).

5.4 La Divergence Fondamentale : Encodage vs. Génération

La conception des modèles basés sur les Transformers reflète une divergence stratégique en TALN. L'encodeur bidirectionnel (BERT) est optimisé pour les tâches discriminatives (analyse, classification, réponse aux questions), car il nécessite une vue complète de la séquence pour déterminer le contexte d'un mot. En revanche, le décodeur unidirectionnel (GPT) est optimisé pour la génération séquentielle, imitant la manière dont le langage est naturellement produit.

Cette divergence est cruciale sur le plan industriel : tandis que BERT est la référence pour l'analyse fine (classification, NER), GPT est le moteur des interfaces conversationnelles et de la création de contenu.

6 Services d'intelligence artificielle pour de l'embedding

Parmi les services d'intelligence artificielle permettant de faire de l'embedding de textes, on retrouve l'API de **Mistral AI** (avec ses modèles comme 'mistral-embed'), **Google Cloud Vertex AI** ('text-embedding-004'), ainsi que la plateforme **Hugging Face** qui offre à la fois une Inference API pour des modèles pré-entraînés et un accès direct à des modèles open source comme ceux de la bibliothèque Sentence Transformers (par exemple 'all-MiniLM-L6-v2' ou 'bge' series).

A noter que certains de ces services sont payants ou en accès limités, citons par exemple :

- OpenAi (payant)
- Google Cloud Vertex AI (accès limité)
- Mistral (accès limité)

6.1 Mistral

Nous utilisons l'API Mistral pour faire de l'emnbedding. Il nous suffit de créer un compte sur le site Mistral.ai, de générer une clé API, nous pouvons ensuite requêter l'API Mistral et faire de l'embedding de textes.

```
import os
from mistralai import Mistral
from dotenv import load_dotenv

load_dotenv()
client = Mistral(api_key=os.getenv('MISTRAL_API_KEY'))

texts = ["Bonjour, ceci est un exemple de texte", "Un deuxieme texte a
    encoder"]

response = client.embeddings.create(
    model="mistral-embed",
    inputs=texts
)

embeddings = [embedding.embedding for embedding in response.data]
print(f"Nombre d'embeddings : {len(embeddings)}")
print(f"Dimension d'un embedding : {len(embeddings[0])}")
```

Type : API cloud payante
Modèles : mistral-embed (1024D), mistral-embed-v2 (2048D)
Tarifs : 0.10−0.50/million tokens
Latence : Très rapide (50-100ms)

6.2 Cohere

```
import os
import cohere
from data import SENTENCES
from benchmark_utils import benchmark
from dotenv import load_dotenv
import numpy as np

load_dotenv()
```

```python

cohere_api_key = os.getenv('COHERE_API_KEY')
co = cohere.Client(cohere_api_key)

texts = ["Bonjour, ceci est un texte en francais",
         "Hello, this is an English text",
         "Hola, este es un texto en espanol"]

response = co.embed(
    texts=texts,
    model="embed-multilingual-v3.0",
    input_type="search_document"      # Type d'utilisation
)

embeddings = response.embeddings
print(f"Nombre de textes : {len(embeddings)}")
print(f"Dimension d'un embedding : {len(embeddings[0])}")
print(f"Premier embedding (10 premieres valeurs) : {embeddings[0][:10]}")

# Conversion en numpy array pour manipulation
embeddings_array = np.array(embeddings)
print(f"\nShape du tableau d'embeddings : {embeddings_array.shape}")
```

Type : API cloud premium
Modèles : embed-multilingual-v3.0, embed-english-v3.0
Tarifs : 0.10−0.40/million tokens
Spécialité : Embeddings optimisés pour la recherche

6.3 Hugging Face

```python
import requests

API_URL = "https://api-inference.huggingface.co/pipeline/feature-extraction/sentence-transformers/all-MiniLM-L6-v2"
headers = {"Authorization": f"Bearer hf_LEoeJZgEAkbqPhXXXXXXXXXXQQNIhNUTbDlYPZIXZ"}

def get_embeddings(texts):
    response = requests.post(API_URL, headers=headers, json={"inputs": texts})
    return response.json()

texts = ["Bonjour, ceci est un texte", "Hello, this is a text"]
embeddings = get_embeddings(texts)
print(f"Nombre d'embeddings : {len(embeddings)}")
```

Type : Bibliothèque Python locale
Coût : 100
Modèles : Tous modèles publics
Installation : pip install sentence-transformers

Inconvénients :
Requiert du CPU/GPU local
Installation initiale
Moins performant que les modèles propriétaires.

Dans mon projet de société au Crédit Agricole, nous utiliserons le modèle **...** avec sentence-transformers.

6.4 OpenRouter

OpenRouter.ai
OpenRouter est un agrégateur d'APIs LLM — il donne accès à plusieurs modèles (dont certains gratuits) par une seule interface, on peut ainsi comparer facilement différents services et différentes tâches liées au LLM en ne gérant qu'une seule clé API.

Exemple d'utilisation de **OpenRouter** avec le modèle **thenlper/gte-base"**

```
import os
import requests
import json
from dotenv import load_dotenv

load_dotenv()
openrouter_api_key = os.getenv('OPENROUTER_API_KEY')

response = requests.post(
  url="https://openrouter.ai/api/v1/embeddings",
  headers={
    "Authorization": f"Bearer {openrouter_api_key}",
    "Content-Type": "application/json",
  },
  data=json.dumps({
    "model": "thenlper/gte-base",
    "input": "Your text string goes here",
    "encoding_format": "float"
  })
)

print(f"Longueur de l'emmbedding: {len(response.json()['data'][0]['embedding
    '])}")
```

Quelques modèles disponibles d'embedding sur OpenRouter :

- thenlper/gte-base, thenlper/gte-large
- intfloat/e5-large-v2, intfloat/e5-base-v2, intfloat/multilingual-e5-large
- sentence-transformers/paraphrase-minilm-l6-v2, sentence-transformers/all-minilm-l12-v2, sentence-transformers/multi-qa-mpnet-base-dot-v1, sentence-transformers/all-mpnet-base-v2, sentence-transformers/all-minilm-l6-v2
- baai/bge-base-en-v1.5, baai/bge-large-en-v1.5, baai/bge-m3
- mistralai/mistral-embed-2312, mistralai/codestral-embed-2505
- google/gemini-embedding-001
- openai/text-embedding-ada-002, openai/text-embedding-3-large openai/text-embedding-3-small
- qwen/qwen3-embedding-8b, qwen/qwen3-embedding-4b, qwen/qwen3-embedding-0.6b

7 Évaluation des embeddings

Maintenant que nous avons identifié des services d'IA proposant des modèles d'embedding nous pouvons faire de l'embedding sur des textes et comparer ces embeddings.

7.1 Simple évaluation - Qualité sémantique

7.2 Evaluation académique - Dataset MTEB

Pour évaluer et comparer ces modèles, le benchmark MTEB (Massive Text Embedding Benchmark) s'impose comme référence sectorielle : il évalue les embeddings sur plus de 50 tâches couvrant la recherche sémantique, le clustering, la classification et la similarité textuelle. Des alternatives complémentaires comme BEIR (Benchmarking Embeddings for Information Retrieval) permettent également d'évaluer spécifiquement les performances en recherche documentaire. Ces outils permettent ainsi de sélectionner objectivement le modèle d'embedding le plus adapté à votre cas d'usage, en croisant les performances techniques avec les contraintes d'intégration via API.
Pour évaluer nos différentes méthodes d'embedding, il nous faut : Un dataset de textes, des tâches de traitement naturel du langage ainsi que des modèles pour effectuer ces différentes tâches.

Les tâches classiques pour le traitement du langage naturel sont :

- Questions / Réponses
- Traduction
- Résumé de textes
- Classification de textes
- Clustering de textes
- Recherche d'informations

7.3 Les Benchmark MTEB

Le benchmark le plus complet à ce jour est MTEB (Muennighoff et al., 2022). Ce "Massive Text Embedding Benchmark" regroupe une cinquantaine de jeux d'évaluation ciblés sur 8 tâches de TALN, mais il présente une limite majeure : il est principalement concentré sur l'anglais. Certaines initiatives ont déjà étendu le benchmark à d'autres langues, comme le chinois (Xiao et al., 2024) et l'allemand (Wehrli et al., 2024). Nos travaux s'inscrivent dans la même direction avec pour cible le français

7.4 Les Benchmarks Standards : GLUE et SuperGLUE

Pour standardiser l'évaluation extrinsèque, des benchmarks synthétiques ont été créés :GLUE (General Language Understanding Evaluation) : Ce benchmark initial offrait une métrique unique pour résumer les progrès sur neuf tâches diversifiées de compréhension du langage.

SuperGLUE : L'efficacité rapide des modèles basés sur les Transformers, qui ont rapidement approché ou dépassé les performances des humains non-experts sur GLUE, a nécessité la création de SuperGLUE.

SuperGLUE propose un nouvel ensemble de tâches de compréhension plus difficiles, exigeant des capacités de raisonnement contextuel plus sophistiquées.19Le remplacement rapide de GLUE par SuperGLUE démontre l'accélération exponentielle des progrès dans l'encodage contextuel. Cela révèle que les modèles modernes maîtrisent largement les tâches sémantiques

simples (comme les analogies) et que l'évaluation doit désormais se concentrer sur des tâches extrinsèques complexes (raisonnement, inférence) pour mesurer la véritable compréhension générale du langage.

Un titre de colonne	Un autre titre	Un dernier
à gauche	centré	à droite
d	e	f

FIGURE 8 – *Un tableau simple*

Tableau Comparatif des Architectures d'EncodageÈre/ModèleNature de la Représentation-Mécanisme Contexte/SémantiqueRésolution de la PolysémieI. BoW/TF-IDFCreuse (Sparse)Fréquence statistiqueNulleII. Word2Vec/GloVeDense (Statique)Prédiction/Co-occurrence locale ou globaleFaible (un seul vecteur)II. FastTextDense (Statique/Subword)N-grammes de caractèresFaibleIII. BERT (Transformer)Dense (Dynamique)Auto-attention BidirectionnelleExcellente (vecteur par instance)III. GPT (Transformer)Dense (Dynamique)Auto-attention UnidirectionnelleBonne (vecteur par instance)

8 Défis Opérationnels, Éthiques et Perspectives d'Avenir

8.1 Le Défi du Coût de Calcul

L'évolution vers l'encodage contextuel de haute fidélité s'est accompagnée d'une augmentation spectaculaire des ressources requises. La construction d'un Grand Modèle de Langage (LLM) à partir de zéro, tel que le GPT-3 (estimé à 175 milliards de paramètres), nécessite une puissance de calcul colossale, des ressources financières importantes et beaucoup de temps. Cette barrière de coût limite l'accès à la création de modèles fondateurs à quelques organisations disposant de ressources massives.

Pour adresser ce problème, notamment lors de l'inférence, des modèles optimisés ont été développés. Par exemple, DistilBERT réduit la taille et le temps d'inférence de BERT tout en maintenant une grande partie de ses performances, améliorant ainsi la productivité et réduisant le coût opérationnel.

8.2 Défis Éthiques : La Propagation des Biais

Les modèles d'encodage vectoriel sont entraînés sur d'énormes volumes de données textuelles issues d'Internet. Ces corpus intègrent et amplifient les préjugés et les stéréotypes sociaux existants (liés au genre, à la race, etc.).

L'impact de ces préjugés est significatif, affectant potentiellement les utilisateurs du modèle et la société dans son ensemble, en conduisant à des résultats discriminatoires ou injustes dans des applications critiques. Les biais sont encodés directement dans la structure de l'espace vectoriel (par exemple, des associations stéréotypées entre métiers et genres).

Le coût exorbitant du pré-entraînement mène à une concentration du développement des LLMs fondamentaux. Cela signifie qu'un petit nombre d'organisations détermine, par leurs choix de données et d'architecture, la structure sémantique et les biais inhérents qui seront répercutés dans l'espace vectoriel utilisé par des milliers d'applications tierces. L'atténuation de ces biais nécessite des efforts de débiaisement des données d'entraînement et des ajustements des pondérations du modèle.

8.3 Tendances Futures : Les LLMs Multimodaux

L'avenir de l'encodage vectoriel se dirige vers la multimodalité, où les Large Multimodal Models (LMMs) intègrent le texte avec d'autres modalités de données, principalement la vision (images et vidéo) et l'audio.

7.3.1. Architectures Clés Des modèles pionniers comme CLIP (Contrastive Language-Image Pre-training) utilisent l'apprentissage contrastif pour aligner les espaces vectoriels du langage et de l'image, permettant au modèle d'associer des concepts textuels à des représentations visuelles. D'autres, comme Flamingo, combinent un encodeur visuel dédié avec un modèle de langage.

7.3.2. Applications Les LMMs ouvrent des possibilités d'innovation quasi illimitées. Les applications concrètes incluent :

La génération d'images à partir de descriptions textuelles.

L'analyse en temps réel de vidéos, permettant de déduire des émotions ou des actions basées à la fois sur le dialogue et les expressions visuelles.

Le marketing personnalisé, synchronisant les vecteurs de texte et les visuels pour créer des publicités ciblées.

La convergence des capacités de traitement du langage naturel et d'analyse d'image transforme profondément l'interaction entre l'homme et la machine.

9 Conclusion

Conclusion
1. Récapitulatif de l'Évolution : Du simple comptage de symboles (Bag-of-Words) à la capture de sens statique (Word Embeddings) puis à la compréhension contextuelle profonde (Transformers). 2. L'État de l'Art : Aujourd'hui, les encodeurs contextuels basés sur les Transformers sont la référence pour la plupart des tâches de NLP, car ils capturent la nuance et la polysémie du langage. 3. Perspectives Futures : Tendances vers des modèles encore plus grands (LLMs - Large Language Models), plus efficaces, et plus spécialisés. L'encodage multimodal (texte + image) est également un champ de recherche très actif.

10 Divers Infos

Partie 1 : Les Fondations - Méthodes Basées sur le Lexique (Bag-of-Words et variantes)

Principe général : Ces méthodes ignorent l'ordre des mots et se concentrent sur leur présence ou leur fréquence dans un document. Elles créent un "sac" de mots.

1.1. One-Hot Encoding - Concept : Représenter chaque mot par un vecteur binaire de la taille du vocabulaire, où un seul élément est à 1 (celui correspondant au mot) et tous les autres à 0. - Exemple : Vocabulaire = ["chat", "chien", "mange"] -> "chat" = [1, 0, 0], "chien" = [0, 1, 0] - Avantages : Simplicité de mise en œuvre. - Inconvénients : - Vecteurs de très haute dimension (problème de "malédiction de la dimension"). - Aucune notion de similarité entre les mots (tous les vecteurs sont orthogonaux). - Ignore complètement la sémantique et l'ordre des mots.

1.2. Encodage par Comptage (Count Vectorization) - Concept : Étendre le one-hot encoding au niveau du document. Chaque document est représenté par un vecteur où chaque dimension correspond à la fréquence d'un mot du vocabulaire. - Représentation : Matrice Document-Termes (Document-Term Matrix). - Avantages : Simple, conserve une notion d'importance par la fréquence. - Inconvénients : - Les mots très fréquents (et souvent peu informatifs comme "le", "de") dominent la représentation. - Toujours aucune sémantique.

1.3. TF-IDF (Term Frequency-Inverse Document Frequency) - Concept : Amélioration du simple comptage en pondérant la fréquence des termes. - TF (Term Frequency) : Fréquence du terme dans le document. - IDF (Inverse Document Frequency) : Mesure de l'importance du terme dans l'ensemble du corpus. Un mot présent dans tous les documents (ex : "le") a un IDF faible. - Calcul : 'TF-IDF(t, d) = TF(t, d) * log(N / DF(t))' où N est le nombre total de documents. - Avantages : - Réduit l'importance des mots trop communs (stop-words). - Donne plus de poids aux mots "spécifiques" et informatifs d'un document. - Inconvénients : - Ne capture toujours pas la sémantique ni l'ordre des mots. - Performances mitigées pour des tâches complexes comme la traduction ou la synthèse.

Partie 2 : La Révolution Sémantique - Les Word Embeddings (Représentations Denses)

Principe général : Représenter chaque mot par un vecteur dense (de dimension fixe et réduite, ex : 50 à 300) dans un espace vectoriel continu. L'idée clé est que les mots avec des significations similaires ont des vecteurs similaires.

2.1. Principes Fondamentaux des Word Embeddings - Hypothèse de Distribution (Harris, 1954) : "Un mot est caractérisé par les compagnons qu'il fréquente". Les mots apparaissant dans des contextes similaires ont des significations similaires.

2.2. Méthodes Pré-entraînées Classiques - Word2Vec (Mikolov et al., 2013) - Deux architectures : - CBOW (Continuous Bag-of-Words) : Prédit un mot cible à partir de son contexte. - Skip-gram : Prédit le contexte à partir d'un mot cible (généralement plus performant pour les mots rares). - Caractéristiques : Rapide à entraîner, efficace sur de grands corpus.

- GloVe (Global Vectors for Word Representation, 2014) - Concept : Combine les avantages des méthodes de décomposition matricielle (comme LSA) et des méthodes de fenêtre contextuelle locale (comme Word2Vec). - Fonctionnement : Utilise les statistiques globales de co-occurrence des mots sur l'ensemble du corpus. - Avantage : Capture souvent mieux les relations globales que Word2Vec.

- FastText (de Facebook AI, 2016) - Innovation : Représente les mots comme des sacs de caractères n-grammes. Le vecteur d'un mot est la somme des vecteurs de ses n-grammes. - Avantage majeur : Peut générer des vecteurs pour des mots hors-vocabulaire (non vus lors de l'entraînement), ce qui est crucial pour les langues riches en morphologie ou pour gérer les fautes de frappe.

Partie 3 : L'Ère Contextuelle - Les Encodages par Transformeurs

Principe général : Au lieu d'avoir un vecteur fixe pour chaque mot, la représentation d'un

mot dépend du contexte dans lequel il apparaît dans une phrase. Le mot "banque" n'aura pas le même encodage dans "je vais à la banque" et "je m'assois sur la banque".

3.1. La Rupture : L'Architecture Transformer - Introduction de "Attention is All You Need" (Vaswani et al., 2017). - Mécanisme d'Auto-Attention : Permet à chaque mot d'interagir avec tous les autres mots de la phrase pour calculer une représentation contextuelle. - Avantage : Capture les dépendances à longue distance et les relations complexes entre les mots, quel que soit leur position dans la phrase.

3.2. Modèles Pré-entraînés Emblématiques (Pré-training / Fine-tuning) - BERT (Bidirectional Encoder Representations from Transformers) - Innovation : Entraînement bidirectionnel (contrairement à GPT qui est unidirectionnel). Il regarde tout le contexte à gauche et à droite du mot masqué. - Tâches de pré-entraînement : Masquage de mots (MLM) et prédiction de la phrase suivante (NSP). - Utilisation : Génère des embeddings contextuels pour chaque token (mot ou sous-mot) dans une phrase.

- Les Modèles de la Famille GPT (Generative Pre-trained Transformer) - Principe : Architecture de décodeur uniquement, entraînée de manière auto-régressive (prédit le prochain token en ne regardant que les tokens précédents). - Force : Excellents pour la génération de texte.

- Autres Modèles Importants** (à mentionner brièvement) : RoBERTa, XLNet, T5, etc.

3.3. Méthodologie d'Encodage avec les Transformeurs - Tokenization : Découpage du texte en tokens (souvent des sous-mots, ex : "playing" -> "play" + "ing") à l'aide d'algorithmes comme WordPiece (BERT) ou Byte-Pair Encoding (GPT). - Principe d'inférence : Pour obtenir l'encodage d'une phrase, on la passe dans le modèle. La couche de sortie (ou une couche cachée) fournit un vecteur contextuel pour chaque token. - Pooling : Pour obtenir un vecteur pour une phrase/texte entier, on peut moyenner (mean pooling) les vecteurs de tous les tokens, ou utiliser le token spécial '[CLS]' (dans BERT).

Partie 4 : Synthèse Comparative et Guide de Sélection

4.1. Tableau Récapitulatif Comparatif

OOV = Out-Of-Vocabulary (mots non vus lors de l'entraînement)*

4.2. Comment Choisir la Bonne Méthode ? - Tâche simple (ex : classification de sentiments, détection de spam) : TF-IDF ou Word Embeddings statiques (Word2Vec) peuvent suffire et sont peu coûteux. - Tâche complexe (ex : compréhension de question, résumé de texte, traduction) : Les modèles contextuels (BERT et similaires) sont indispensables. - Besoins en ressources (CPU/GPU, temps) : Les méthodes non-contextuelles sont beaucoup plus rapides et moins gourmandes. - Taille du jeu de données : Sur un petit jeu de données, utiliser des embeddings pré-entraînés est crucial. Sur un très grand jeu de données, on peut envisager de fine-tuner un modèle comme BERT. - Langue et domaine : Choisir un modèle pré-entraîné dans la bonne langue et, si possible, sur un domaine proche (ex : BioBERT pour la biomédecine).

10.1 Méthodes basées sur le texte brut

Similarité lexicale

— Jaccard Similarity : Compare le ratio des mots communs entre deux textes. Idéal pour les ensembles de mots (ex : tags, mots-clés).

— Levenshtein Distance : Mesure le nombre d'éditions (ajout/suppression/remplacement) pour passer d'un texte à l'autre. Utile pour la similarité de chaînes courtes (ex : correction orthographique).

10.2 Méthodes statistiques (bag-of-words)

— TF-IDF + Similarité cosinus : Représente les textes comme des vecteurs de poids TF-IDF, puis calcule la similarité cosinus.

— BM25 (Okapi BM25) : Amélioration de TF-IDF pour la recherche d'information, mieux adaptée aux documents longs. Librairie : rank_bm25.

10.3 Méthodes basées sur les embeddings

— Word Embeddings (moyennés) : Moyenne des embeddings des mots (Word2Vec, GloVe, FastText).

— Sentence Embeddings : Sentence-BERT (SBERT) : Modèle dédié pour encoder des phrases.

— Universal Sentence Encoder (USE) : Version allégée disponible via TensorFlow Hub.

—

10.4 Méthodes avec Transformers (modèles contextuels)

— Similarité via [CLS] token (BERT-like) : Utilise le vecteur du token [CLS] de BERT pour représenter le texte.

— Modèles spécialisés en similarité : MPNet, RoBERTa STS : Modèles fine-tunés sur des tâches de similarité sémantique (ex : STSb). Cross-Encoders : Compare deux textes simultanément (précis mais lent).

Quelle méthode choisir ?

Cas d'usage	Méthode recommandée
Texte court, rapidité	Jaccard, TF-IDF + Cosinus
Sémantique profonde	Sentence-BERT, Cross-Encoders
Ressources limitées	Word2Vec/GloVe moyennés
Similarité à grande échelle	FAISS + SBERT

10.5 Divers

BLEU Score (bilingual evaluation understudy) : mesure la qualité d'une traduction de texte.
ROUGE Score (Recall-Oriented Understudy for Gisting Evaluation) : mesure la qualité des résumés automatiques.
Métriques RAGAS

10.6 Principe Sémantique

• Principe sémantique : Le plongement lexical (ou sémantique) représente chaque mot par un vecteur de nombres réels. La technique est basée sur l'hypothèse que les mots qui apparaissent dans des contextes similaires ont des significations apparentées. • Proximité et analogie : La distance et la direction entre les vecteurs numériques reflètent la similarité et les relations entre les mots correspondants. Par conséquent, des mots de sens similaire (comme "chien" et "chat") sont représentés par des vecteurs proches. Cette disposition permet de reproduire des schémas sémantiques et syntaxiques par arithmétique vectorielle, comme l'équation "roi - homme + femme = reine".

Transformer le texte en vecteurs améliore considérablement l'efficacité des modèles d'apprentissage automatique : • Réduction de la dimensionnalité : Les plongements lexicaux sont des vecteurs denses de dimension inférieure. Contrairement aux méthodes traditionnelles qui utilisent des vecteurs creux (comme l'encodage one-hot qui a une dimension égale à la taille du vocabulaire et contient beaucoup de zéros), les plongements réduisent la complexité des calculs et les besoins en mémoire, ce qui est crucial pour les applications de TALN à grande échelle. •

Généralisation : L'approche permet aux modèles de bien généraliser aux mots inconnus ou rares (out-of-vocabulary), car elle leur permet de représenter ces mots en fonction de leur contexte ou de leur structure en sous-mots (comme dans FastText). • Apprentissage accéléré : Les modèles entraînés avec des plongements de mots ont tendance à converger plus rapidement et à être plus performants que ceux utilisant des représentations creuses.

Un bon modèle d'embedding doit être capable de capturer les subtilités sémantiques du langage et de les représenter dans un espace vectoriel de manière à ce que les distances ou similarités entre vecteurs correspondent à des relations sémantiques logiques.

Par exemple, les mots "roi" et "reine" devraient être plus proches entre eux dans l'espace vectoriel que "roi" et "pomme", reflétant leur proximité sémantique dans la langue réelle.

11 Latex Tools

footnote [1]
référence bibliographique [2], [1]

Lien vers section (voir section ...)
Lien vers les métriques RAGAS.

voici une ancre

1. https://keras.io/api/layers/core_layers/embedding/

Références

[1] José Siqueira de Cerqueira, Kai-Kristian Kemell, Rebekah Rousi, Nannan Xi, Juho Hamari, and Pekka Abrahamsson. Mapping trustworthiness in large language models : A bibliometric analysis bridging theory to practice, 2025.

[2] Zihang Li, Yangdong Ruan, Wenjun Liu, Zhengyang Wang, and Tong Yang. Cft-rag : An entity tree based retrieval augmented generation algorithm with cuckoo filter, 2025.